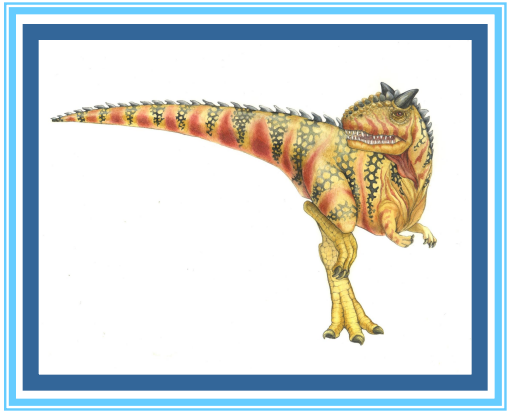
Chapter 1: Introduction



Silberschatz, Galvin and Gagne ©2013



What is an Operating System?

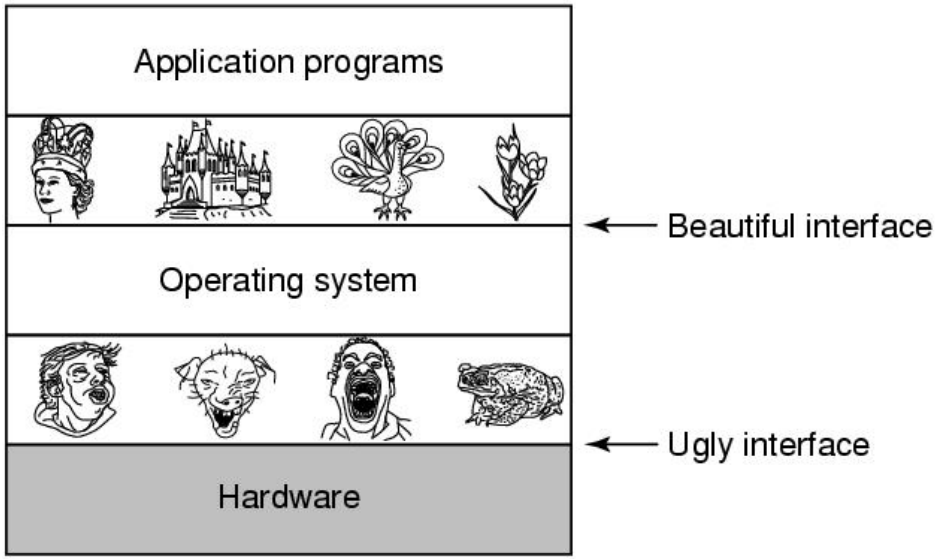
- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Use the computer hardware in an efficient manner





The Operating System as an Extended Machine

An Operating System gives a beautiful interface to the existing hardware system and acts as an Extended Machine.

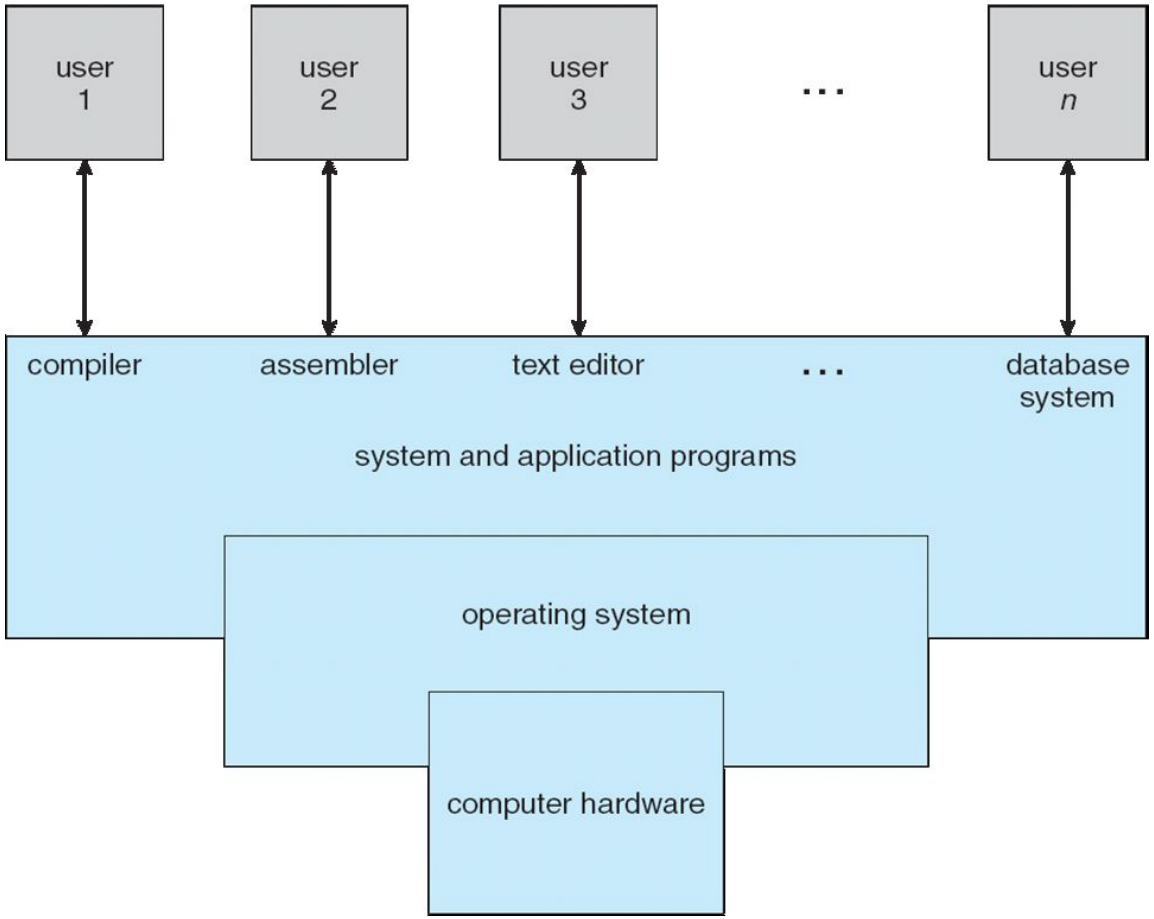


Tanenbaum, Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-6006639





Four Components of a Computer System







Operating System Definition

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer





Operating System Definition (Cont.)

- No universally accepted definition
- "Everything a vendor ships when you order an operating system" is a good approximation
  - But varies wildly
- **"The one program running at all times on the computer" is the kernel.**
- **Part of the operating system that loads first, and it remains in main memory.**
- **The kernel code is usually loaded into a protected area of memory to prevent it from being overwritten by programs or other parts of the operating system.**

Typically, the kernel is responsible for memory management, process and task management, and disk management. The kernel connects the system hardware to the application software.





Operating System Definition (Cont.)

Everything else is either

- **a system program (ships with the operating system) , or**
- **an application program.**





Computer Startup

- **Bootstrap program** is loaded at power-up or reboot
  - Typically stored in ROM or EPROM, generally known as **firmware**
  - Initializes all aspects of system
  - Loads operating system kernel and starts execution





Operating-System Operations (cont.)

- **Dual-mode** operation allows OS to protect itself and other system components
  - **User mode** and **kernel mode**
  - **Mode bit** provided by hardware
    - Provides ability to distinguish when system is running user code or kernel code
    - Some instructions designated as **privileged**, only executable in kernel mode
    - System call changes mode to kernel, return from call resets it to user
- Increasingly CPUs support multi-mode operations
  - i.e. **virtual machine manager** (**VMM**) mode for guest **VMs**





System Calls

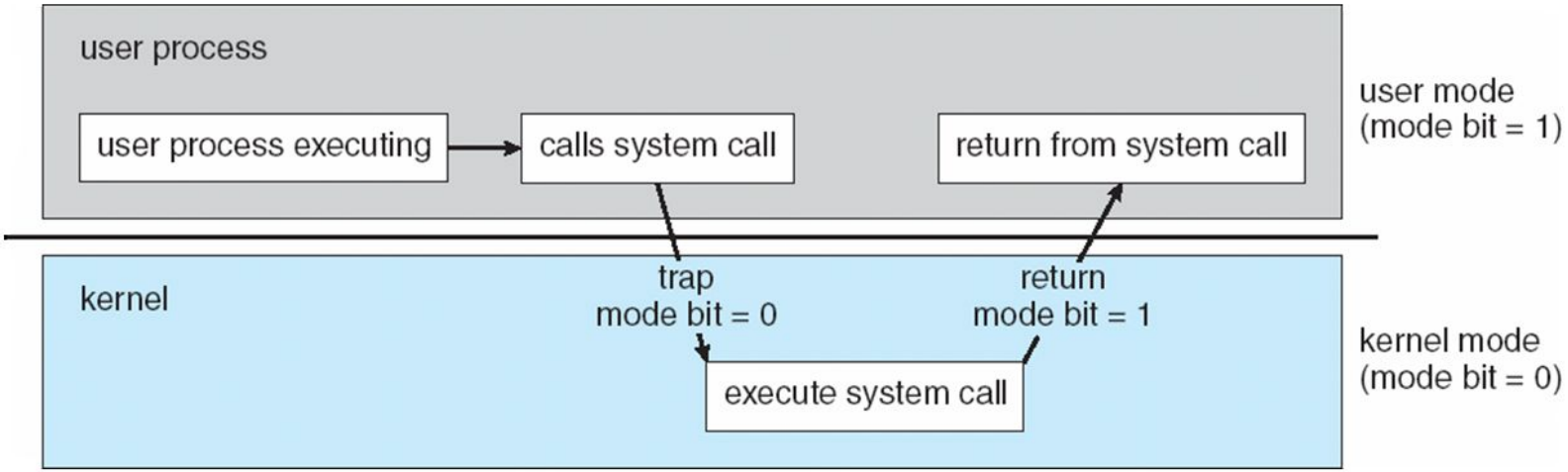
- Interface between user programs and OS
- Varies from OS to OS
- System call issued by user program
- Call uses a library call of the same name
- Library routine puts machine into kernel modes (by issuing a special instruction)
- Finds actual routine for system call in a table
- Does the work involved in the call
- Returns to user program



Tanenbaum Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-600663-9



Transition from User to Kernel Mode







Operating Systems Architecture

Monolithic systems

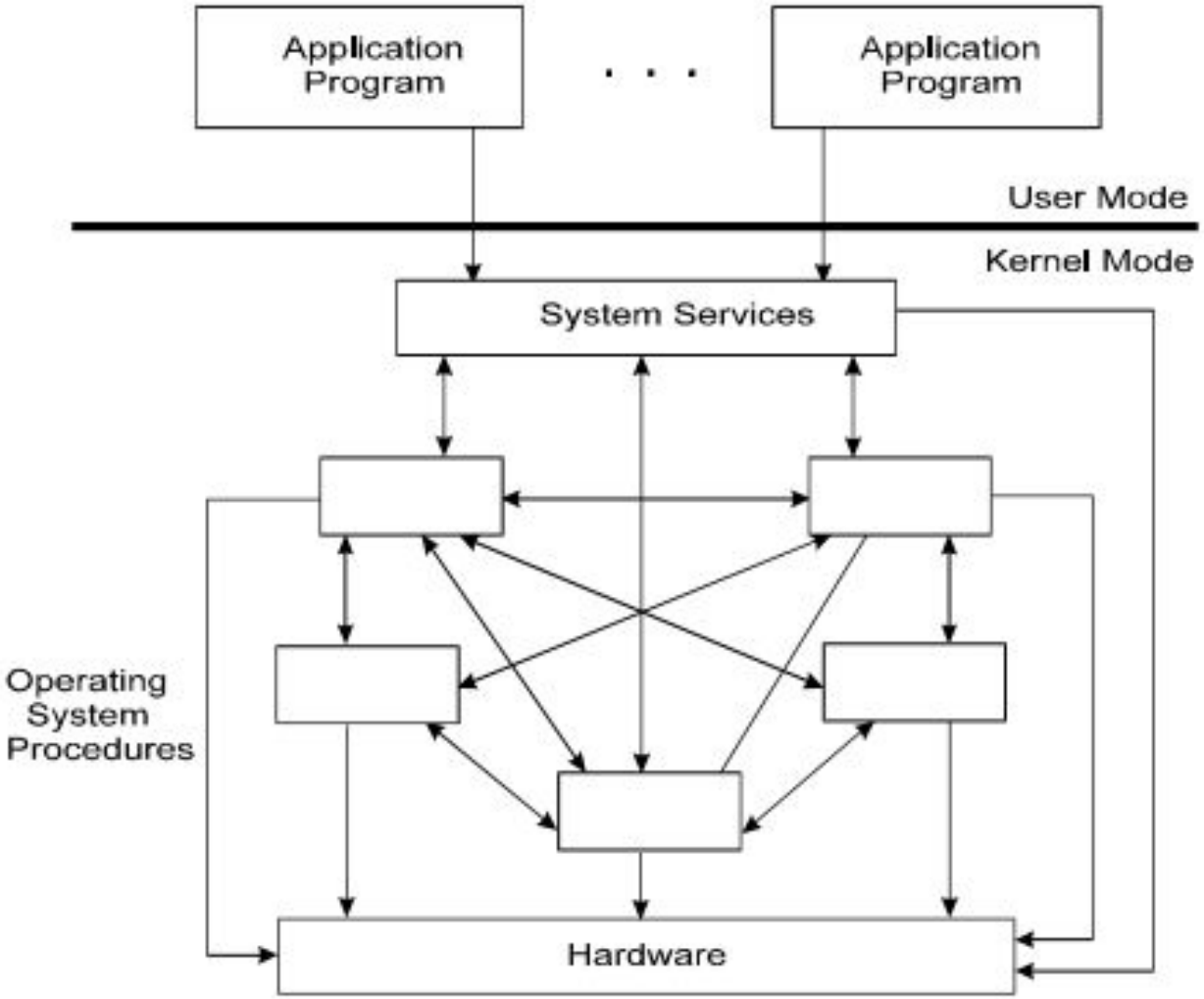
- A main program that invokes the requested service procedure.
- A set of service procedures that carry out the system calls.
- A set of utility procedures that help the service procedures.
- All the basic system services like process and memory management, interrupt handling etc. are packaged into a single module in kernel space.



Tanenbaum Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-600663-9



Monolithic systems







Monolithic systems

Drawbacks

1) Size of kernel is huge.

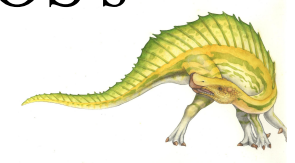
2) Poor maintainability, which means bug fixing or addition of new features resulted in recompilation of the whole kernel which could consume hours

3) A crash in any of these procedures will take down the entire operating system

Modern day approach:

> The kernel consists of different modules which can be dynamically loaded and un-loaded.

> This modular approach allows easy extension of OS's capabilities.





Monolithic systems

- Maintainability of kernel became very easy
  - Only the concerned module needs to be loaded and unloaded every time there is a change or bug fix in a particular module.

> No need to bring down and recompile the whole kernel for a smallest bit of change.

- Stripping of kernel for various platforms became very easy
  - Can easily unload the module that we do not want





Microkernels

- Small number of processes are allowed in the kernel
  - Managing memory protection
  - Process scheduling
  - Inter Process communication (IPC)
- Device driver management, protocol stack, file system etc. may run in the user space (They are called servers and loaded during the time the OS boots-up)
  - Microkernel servers are daemon programs
  - The Kernel grants some of them privileges to interact with parts of physical memory that are otherwise off limits to most programs.
  - This allows some servers, particularly device drivers, to interact directly with hardware.



Tanenbaum Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-600663-9



Microkernels

- Minimizes effects of bugs *Galvin Chapter 2 , 2.7.3
  - Don’t want bug in driver to crash system
  - Ex: Network service crashes due to buffer overflow, then only the networking service's memory would be corrupted, leaving the rest of the system still functional.
- Reduces the kernel code size and increases the security and stability of OS.
- Servers communicate through Inter-process communication / Message passing

<u>Microkernels can suffer from performance decreases due</u>

- <u>to increased system function overhead</u>

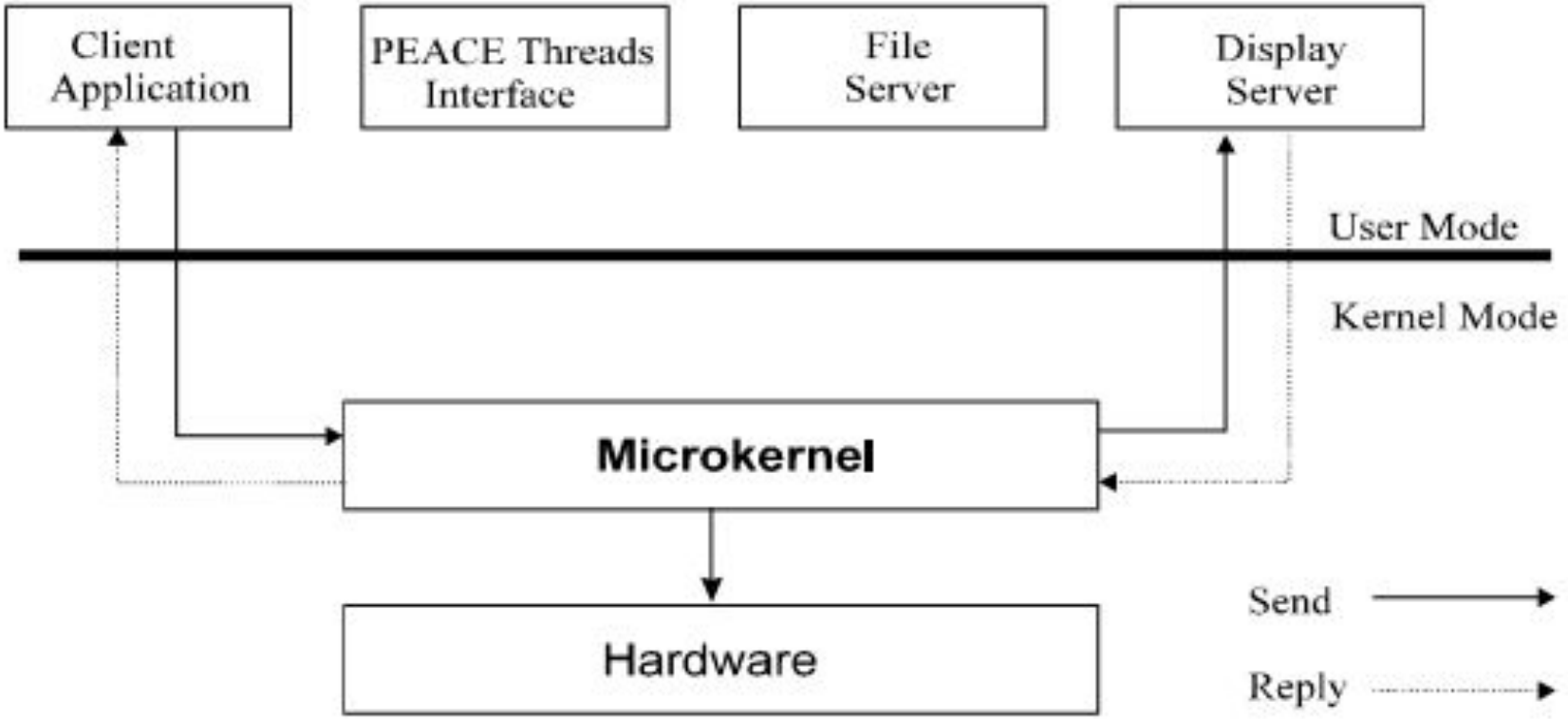
- Ex: Mac OS X, Windows XP, NT, Windows 7 etc.



Tanenbaum Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-600663-9



Microkernels

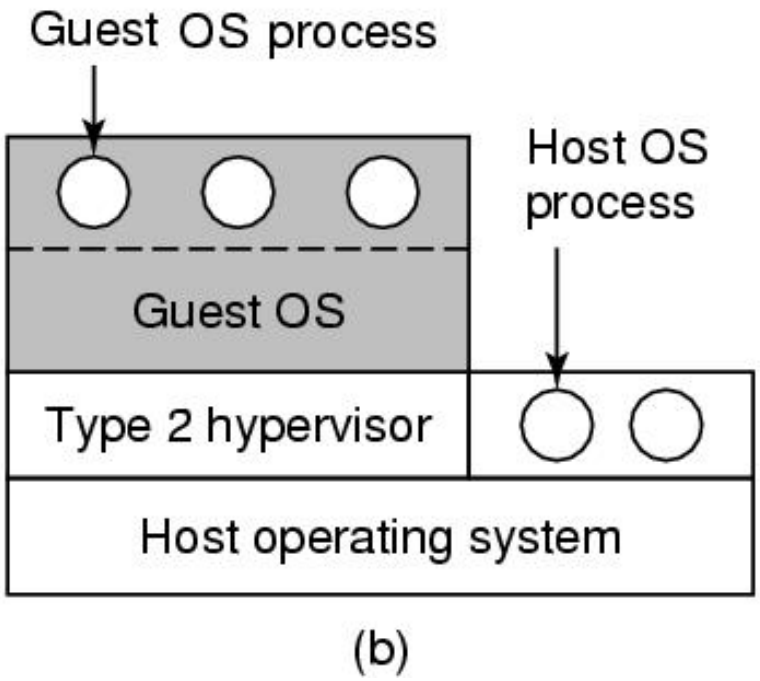




Tanenbaum Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-600663-9



Virtual Machines



(b)

Type 2 hypervisors run on a host operating system. When the virtualization movement first began to take off, Type 2 hypervisors were most popular. Administrators could buy the software and install it on a server they already had.

Examples of this **type** of **hypervisor** include

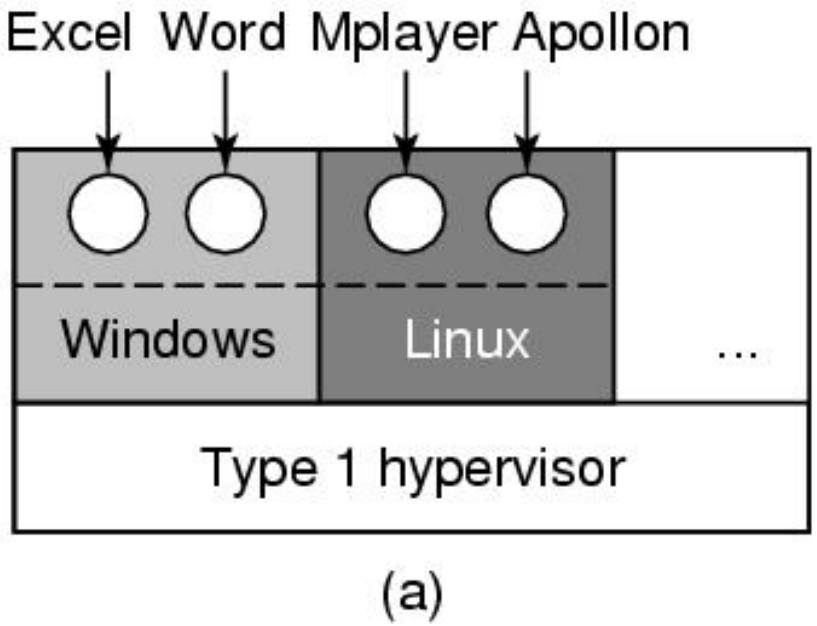
VMware Fusion, Oracle Virtual Box, Oracle VM for x86, Solaris Zones, Parallels and VMware Workstation





Virtual Machines

virtualize the *hardware* of the machine



(a)

Type 1 hypervisors run directly on the system hardware. They are often referred to as a "native" or "bare metal" or "embedded" hypervisors in vendor literature.

is installed directly on physical host server hardware just like an operating system.

Type 1 hypervisors provide higher performance, availability, and security than Type 2 hypervisors

Type 1 hypervisors run on dedicated hardware. They require a management console and are used in data centers. Examples of this type of hypervisor include Oracle OVM for SPARC, ESXi, Hyper-Vand KVM



Tanenbaum Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-600663-9

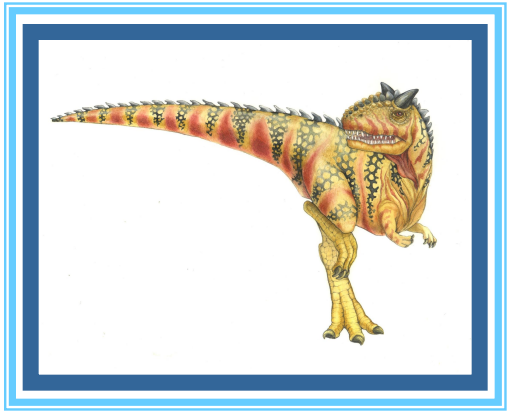


Computing Environments

- Virtualization
- Virtual Desktop Infrastructure



End of Chapter 1



Silberschatz, Galvin and Gagne ©2013